

# Kapitel 13

## Speicher auf dem Heap

### 13.1 Speichertypen

Wir haben bisher zwei logische Arten von Speicher kennengelernt:

- Externe Variablen sind grundsätzlich statisch, ebenso interne Variablen mit dem Attribut `static`. Das hieß, dass bereits der Compiler den erforderlichen Speicherplatz bereitstellt, der dann während der gesamten Ausführung des Programms zur Verfügung steht.
- Automatische Variablen sind dynamisch. Das hieß, dass der Speicher für sie erst in dem Moment bereitgestellt wird, in dem bei der Ausführung des Programms der Block betreten wird, der ihre Definition enthält, und dass dieser Speicher beim Verlassen des Blocks auch wieder freigegeben wird. Diese Art von Variablen wird in der Regel auf einem *Stack* bereitgestellt. Dabei handelt es sich um eine Datenstruktur, die linear wächst. D.h. Speicherplatz einer Variablen auf dem Stack kann erst wieder freigegeben werden, wenn alle später erzeugten Variablen ebenfalls wieder freigegeben wurden. Das entspricht genau der Verwendung von automatischen Variablen. Es handelt sich also um strukturierten dynamischen Speicher.

Es gibt eine dritte logische Art von Speicher: Ein Programm hat die Möglichkeit, während es läuft Speicher vom Betriebssystem anzufordern, wobei sich die Größe des angeforderten Speichers erst bei Ablauf des Programms z.B. durch eine Benutzereingabe ergibt. Dieser Speicher steht ihm dann so lange zur Verfügung, bis es ihn explizit wieder freigibt oder bis es terminiert. Die Verfügbarkeit dieses Speichers ist insbesondere nicht an die Blockstruktur des Programms gebunden. Man bezeichnet die Gesamtheit dieses Speichers als *Heap*. Da dieser Speicher in beliebiger Reihenfolge angefordert und wieder freigegeben werden kann, ist klar, dass er nicht besonders strukturiert sein kann. Die Funktionen zur Anforderung von Speicher auf dem Heap müssen also einen gewissen Aufwand betreiben, um den Speicher zu verwalten. Der von automatischen Variablen genutzte Speicher auf dem Stack erfordert fast keinen zusätzlichen Aufwand.

Wieviel Speicher auf dem Heap zur Verfügung steht, hängt vom jeweiligen Rechner und Betriebssystem ab. Die Größe des Heap kann durch den physikalisch vorhandenen Speicher begrenzt sein. Die meisten Betriebssysteme unterstützen jedoch virtuellen Speicher. D.h. jedem Programm steht der gesamte Adressraum zur Verfügung. Sollte mehr als der tatsächlich vorhandene Speicher benötigt werden, so werden gerade nicht benötigte Speicherbereiche auf die Festplatte ausgelagert. Belegt ein Zeiger 32 Bit, so sind theoretisch maximal 4 Gigabyte adressierbar.

Bei statischem Speicher und bei Speicher auf dem Stack sorgt der Compiler dafür, dass eine Zuordnung zwischen Variablen und Adressen erfolgt. Bei Speicher auf dem Heap sieht das ganz anders aus: Die Anforderung von Speicher vom Betriebssystem erfolgt natürlich durch Aufruf einer entsprechenden Funktion. Die kann – als einzige Möglichkeit – einen Zeiger auf den Anfang des bereitgestellten Speichers als Funktionswert zurückgeben, falls genug Speicher vorhanden ist. Man ist also gezwungen, diesen Wert in einer „passenden“ Zeigervariablen zu speichern, damit man ihn später für die Zugriffe auf den Speicher verwenden kann.

## 13.2 Funktionen zur Heapverwaltung

C kennt drei Funktionen, mit denen man Speicher auf dem Heap anfordern kann, und eine Funktion zur Freigabe. Alle vier Funktionen sind in der Standard-Headerdatei `stdlib.h` deklariert. Wichtig sind zunächst nur die Funktionen

```
void *malloc(size_t groesse);
void free(void *zeiger);
```

Die Funktion `malloc` stellt, wenn möglich, einen zusammenhängenden Speicherbereich der Größe `groesse` zur Verfügung und liefert als Funktionswert den Zeiger auf den Anfang dieses Speicherbereichs. Die Größe bezieht sich dabei auf ein Vielfaches des Speicherbedarfs eines Zeichens – wir erinnern uns: Der Standard schreibt `sizeof (char) == 1` vor. Falls der angeforderte Speicher nicht bereitgestellt werden kann, ist der Funktionswert der Nullzeiger (`NULL`). Da die Speicherbereitstellung bei jedem Versuch fehlschlagen kann, sollte man den Rückgabewert von `malloc` *immer* auf `NULL` überprüfen.

Der Typ dieser Funktion ist `void *`. Ein Zeiger dieses Typs ist kompatibel zu allen anderen Zeigertypen, so dass man seinen Wert jeder beliebigen Zeigervariablen zuweisen kann, unabhängig von deren Typ und ohne einen Castoperator hinschreiben zu müssen. Dass man Zeiger mit dem Typ `void *` selbst nicht dereferenziert werden kann, haben wir bereits gesehen.

Die Funktion `free` gibt einen Speicherbereich wieder frei. Voraussetzung für ordnungsgemäßes Arbeiten der Funktion ist, dass man ihr nur Zeiger als Argument übergibt, die man sich zuvor mit `malloc` oder einer der beiden anderen Bereitstellungsfunktionen beschafft hat. Dahinter steht, dass zumindest manche Betriebssysteme „in der Nähe“ der Zeiger, die sie beim Aufruf von `malloc` liefern, zusätzliche interne Informationen unterbringen, die beim Aufruf von `free` benötigt werden, etwa Angaben über die Größe des Speichers. Einzige Ausnahme ist `NULL`. Wird `NULL` als Argument von `free` verwendet, tut die Funktion nichts.

## 13.3 Bereitstellung von Speicher

Bei der Matrizenmultiplikation, oder auch vielen anderen mathematischen Programmen, die mit Vektoren und Matrizen arbeiten, weiß man in der Regel beim Schreiben des Programms nicht, wie groß die Vektoren und Matrizen maximal werden können. Dies macht die Verwendung von statischen Variablen problematisch.

Dieses Problem wird in C durch die Bereitstellung der benötigten Vektoren und Matrizen auf dem Heap gelöst. Durch

```
float *v;
int laenge;
/* ... Initialisierung von laenge */
v = (float *) malloc(laenge * sizeof(float));
```

```

if (v == NULL) {
    /* Fehlerbehandlung, ggf. Programmabbruch */
}

```

stellt das Betriebssystem ja gerade den Speicherplatz bereit, den ein `float`-Vektor mit `laenge` Komponenten benötigt. Entsprechend kann man sich mit

```

float *m;
int n;
/* ... Initialisierung von n */
m = (float *) malloc(n * n * sizeof(float));
if (m == NULL) {
    /* Fehlerbehandlung, ggf. Programmabbruch */
}

```

den Speicherplatz für eine  $(n \times n)$ -Matrix mit `float`-Komponenten beschaffen.

Zeigervariablen erlauben es immer, ihren Wert als Zeiger auf den Anfang eines Vektors zu interpretieren. Das haben wir hier für den Speicher auf den Heap genutzt. Ohne weiters ist es dagegen nicht möglich, Speicher auf dem Heap mit einer Matrixstruktur zu versehen. Wir sind also, wie in der Regel auch in Funktionen, dazu gezwungen, die *Speicherabbildungsfunktion* selber zu definieren und sie für jeden Zugriff selber auszuwerten. Die entsprechende Vorgehensweise wurde schon im Abschnitt über Funktionen besprochen.

Am Ende dieses Kapitels wird noch einmal auf Matrizen auf dem Heap eingegangen. Wir werden sehen, dass sich, wenn auch mit einigem Aufwand, letztlich doch auch Matrizen erzeugen lassen.

Speicher, den man sich mit `malloc` beschafft, besitzt übrigens keine wohldefinierten Werte. Will man bestimmte Anfangswerte haben, muss man selber explizit für deren Zuweisung sorgen.

## 13.4 Beispiel: Speichern von Zahlen

Es wird Zeit für ein konkretes Beispiel. Wir wollen nur die letzten Zahlen einer Eingabe verarbeiten, zum Beispiel die letzten 10. Allerdings soll die Anzahl letztlich erst beim Aufruf des Programms festgelegt werden: Wenn ein entsprechender Aufrufparameter (in der Form `-nnn`) angegeben ist, soll sein Wert verwendet werden; sonst soll 10 verwendet werden.

Um das Programm nicht ausufern zu lassen, werden zwei Vereinfachungen gegenüber der „Soll-Lösung“ vorgenommen:

- Unzulässige Aufrufparameter sollen nicht besonders bemängelt, sondern nur durch den Standardwert 10 ersetzt werden.
- Die Verarbeitung soll nur im Auflisten der Zahlen bestehen.

Die Lösung verwendet die Funktion `sscanf`, die in `string.h` deklariert wird. Sie arbeitet genau wie `scanf`, jedoch liest sie nicht von der Standardeingabe, sondern verarbeitet als Eingabe den String, auf den das erste Argument zeigt.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define DEFLEN 10

```

```

/** Prototypen *****/
static int lesen(unsigned int maxlen, float *v);

/*= Hauptprogramm =====*/
int main(int anzahl, char *argument[]) {
    unsigned int maxlen, len, i;
    float *v;

    if (anzahl < 2 || sscanf(argument[1], "%u", &maxlen) != 1)
        maxlen = DEFLen;

    v = (float *) malloc(maxlen * sizeof (float));
    if (v == NULL) {
        printf("Speicher nicht verfuegbar!\n");
        return 1;
    }

    printf("Bitte Zahlen eingeben: ");
    len = lesen (maxlen, v);

    for (i = 0; i < len; i++)
        printf("%f\n", v[i]);

    free(v);
    return 0;
}

/*= Lesen der Eingabe =====*/
static int lesen(unsigned int maxlen, float *v) {
    int i = 0;
    float z;

    while (i < maxlen) {
        if (scanf("%f", &v[i]) == EOF)
            return i;
        i++;
    }

    while (scanf("%f", &z) != EOF) {
        memmove(&v[0], &v[1], (maxlen - 1) * sizeof (float));
        v[maxlen - 1] = z;
    }

    return maxlen;
}

```

Quelltext 13.1: Auflisten der letzten Zahlen der Eingabe

Eine Frage stellt sich noch: Ist es eigentlich zweckmäßig, die gespeicherten Werte umzuspeichern? Lässt sich dieses ggf. vermeiden?

Zur ersten Teil der Frage: Ganz ohne Umspeicherungen kommen wir nicht aus. Da **float**-Werte in der Regel nur 4 Bytes beanspruchen, spricht im Beispiel wenig dagegen, sie selbst umzuspeichern – **short**- und **char**-Werte benötigen zwar in der Regel weniger Speicherplatz, aber diese Typen helfen hier nicht weiter.

Anders sieht es aus, wenn die Werte mehr Speicherplatz benötigen – etwa wenn sie Strings

oder, allgemeiner, Vektoren sind. Das führt zum zweiten Teil der Frage: Natürlich brauchen wir die Werte selbst nicht umzuspeichern! Um das Umspeichern zu vermeiden, müssen wir allerdings die Datenstruktur ändern: Anstelle eines Vektors, der selbst die Daten enthält, müssen wir einen Vektor definieren, der Zeiger auf die Daten enthält. Dann reicht es aus, die Zeiger umzuspeichern – und die belegen in der Regel jeweils 2 oder 4 Bytes, je nach Rechner.

Realisiert man diese Idee, so hat man zwei Schritte auszuführen: Im ersten muss man den Zeigervektor bereitstellen und im zweiten die Zeiger auf die Daten. Und dabei kann man sich geschickt oder ungeschickt anstellen: Ungeschickt wäre es, in einer Schleife für jede Vektorkomponente separat den Zeiger auf einen passenden Speicherbereich zu beschaffen, weil `malloc` in der Regel eine ziemlich aufwendige Funktion ist. Es reicht durchaus aus, einen einzigen, zusammenhängenden Speicherbereich zu beschaffen und dann Zeiger auf Teile dieses Speicherbereichs zu berechnen. Das können wir, wenn wir beim Werttyp `float` bleiben, zum Beispiel so lösen:

```
float **v, *z;
int j;
...
v = (float **) malloc(maxlen * sizeof(float *));
if (v != NULL) {
    z = (float *) malloc(maxlen * sizeof(float));
    if (z != NULL) {
        for (j = 0; j < maxlen; j++)
            v[j] = &z[j];
    }
    else {
        free(v);
        v = NULL;
    }
}
```

Jetzt zeigt die  $j$ -te Komponente von `v` anfangs auf die  $j$ -te Komponente von `z` – doch das ändert sich unter Umständen während der Ausführung des Programms.

In der Folge sind weitere Änderungen am Programm erforderlich. Sie sollten sie sich selbst einmal überlegen.

## 13.5 Speicherfreigabe

Am Beginn des Abschnitts wurde erwähnt, dass Speicher auf dem Heap dem Programm so lange zur Verfügung steht, bis es ihn wieder freigibt. Das stimmt, leider, nur bei formaler Betrachtung: Der Speicher steht zwar formal zur Verfügung – um ihn nutzen zu können, braucht man zusätzlich aber auch einen Zeiger auf ihn. Und diese Zeiger können in der Regel selbst nicht auf dem Heap liegen, sondern müssen statische oder automatische Variablen sein.

Wir sehen uns das an einem Beispiel an: Formal ist die Funktion

```
void muellerzeugung(int menge) {
    float *v;
    v = (float *) malloc(menge * sizeof(float));
}
```

sicher korrekt. Allerdings verschwindet ihre lokale, automatische Variable `v` mit Abschluss der Funktion wieder – und mit ihr auch der Zeiger auf den Speicherbereich, den wir uns mit `malloc` beschafft haben.

Mit einer Funktion wie `muellerzeugung` können wir entsprechend jedes Programm zum „Absturz“ bringen – wir brauchen sie nur oft genug mit einem hinreichend großen Argument aufzurufen: Bei jedem Aufruf wird Speicher angefordert. Da dieser Speicher nicht wieder freigegeben wird, muss der Heap notwendig irgendwann voll sein.

Es ist also ein schwerwiegender Fehler, Speicher auf dem Heap zu beschaffen, diesen aber nicht wieder freizugeben. Wird der Speicher nur lokal in einer Funktion benötigt, so ist diese auch dafür zuständig, ihn wieder freizugeben. Das kann so aussehen:

```
void kein_muell (int menge) {
    float *v;
    v = (float *) malloc(menge * sizeof(float));
    /* ... */
    free (v);
}
```

Oft soll aber der bereitgestellte Speicher über einen längeren Zeitraum verwendet werden. Dann sollte es möglich sein, die Speicherreservierungen zu protokollieren, um während der Testphase des Programms eventuelle Speicherlecks mithilfe des Protokolls zu entdecken. Der Protokollmechanismus kann so implementiert werden, dass man ihn z.B. durch bedingte Compilation leicht abschalten kann. Damit reservierter Speicher leicht wieder freigegeben werden kann, sollte es zu jeder Funktion, die Speicher bereitstellt, eine passende Funktion geben, die ihn wieder freigibt. Besonders bei komplexen Datenstrukturen kann das sehr hilfreich sein: Man muss am Ende nur zählen, ob beide Funktionen gleichzeitig erfolgreich aufgerufen wurden. Im nächsten Abschnitt wird für Matrizen ein solchen Funktionenpaar vorgestellt.

Außerdem gilt natürlich erneut, was schon für Zeiger auf Konstanten gilt: Überschreibt man den Zeiger auf einen Speicherbereich im Heap durch einen anderen Wert, hat man anschließend keinen Zugriff mehr auf ihn. Insbesondere kann man ihn nicht einmal mehr freigeben!

So offensichtlich wie in der Anweisungsfolge

```
float f, *v;
v = (float *) malloc(laenge * sizeof(float));
v = &f;
```

wird das sicher kein Programmierer machen – in verdeckter Form kommt es leider allzu häufig vor!

## 13.6 Matrizen auf dem Heap

Jetzt soll endlich ein allgemein brauchbares Beispiel für beliebig dimensionale Matrizen implementiert werden. Wie man sich den Speicher für sie auf dem Heap beschaffen kann, haben wir oben gesehen. Wie man eine vernünftige Indizierung erreichen kann, wollen wir uns jetzt überlegen.

Dazu müssen wir uns noch einmal in Erinnerung rufen, wie für eine Matrix `m` der Ausdruck `m[i]` zu interpretieren ist. Zunächst zwei elementare formale Überlegungen:

- Paare eckiger Klammern sind Operatoren; aufeinanderfolgende Paare werden von links nach rechts abgearbeitet.
- Der Name einer Matrix repräsentiert einen Zeiger auf den Vektor der Zeiger auf die Zeilen.

Beide Überlegungen ergeben zusammen: Der Wert des Ausdrucks `m[i]` ist ein Zeiger auf eine Zeile der Matrix.

Da `m` der Name einer Matrix ist, ist der Vektor der Zeiger auf die Zeilen und entsprechend der Wert des Ausdrucks `m[i]` eine Konstante. Es steht allerdings nirgends, dass bei doppelter Indizierung, etwa `m[i][j]`, der Teilausdruck `m[i]` eine Konstante sein *muss*.

Und dieses können wir nutzen: Wir wollen einmal unterstellen, dass wir eine  $n \times m$ -Matrix *A* mit `double`-Komponenten benötigen. Dann beschaffen wir uns in einem ersten Schritt einen Vektor mit *n* Komponenten, die jeweils den Zeiger auf eine Zeile aufnehmen können. In einem zweiten Schritt beschaffen wir uns die Speicher für die Zeilen und tragen die Zeiger darauf in die Komponenten des zunächst beschafften Vektors ein.

Im Programm kann das so aussehen:

```
int n, m;
...
double **a;
int i;
...
a = (double **) malloc(n * sizeof (double *));
if (a == NULL)
    /* ... Fehlerbehandlung */

for (i = 0; i < n; i++) {
    a[i] = (double *) malloc(m * sizeof (double));
    if (a[i] == NULL)
        /* ... Fehlerbehandlung */
}
```

Schreiben wir jetzt etwa `a[i][j]`, so wird wieder zunächst `a[i]` ausgewertet. Dabei resultiert der Zeiger auf die entsprechende Zeile. Danach erfolgt die zweite Indizierung und liefert uns den Wert der entsprechenden Komponente der Zeile, also genau das, was wir haben wollen.

Bei der Freigabe einer so bereitgestellten Matrix muss man offensichtlich genau umgekehrt vorgehen und erst die einzelnen Zeilen und dann den Vektor der Zeiger auf die Zeilen freigeben. Das Arbeiten mit Matrizen wird dadurch sehr einfach, da man so bereitgestellte Matrizen auch in Funktionen vernünftig indizieren kann. Die Parameter, die die Matrizen repräsentieren, sind formal ja Vektoren (von Zeigern) – und für Vektoren funktioniert die Indizierung auch in Funktionen.

Der Preis, den man dafür zahlen muss, sind die relativ aufwendige Bereitstellung und Freigabe der Matrizen. Man kann ihn aber auf ein Minimum reduzieren, indem man einmal die entsprechenden Operationen als Funktionen programmiert und in einem separaten Modul unterbringt, auf den man dann bei Bedarf zurückgreifen kann.

Wenn man diese Funktionen schreibt, kann man gleich noch etwas optimieren: Die Funktionen `malloc` und `free` sind in der Regel ziemlich aufwendig, so dass man sie nicht unnötig oft aufrufen sollte. Entsprechend ist es hier in der Regel günstiger, sich den gesamten Speicherbereich für die Matrix mit einem Aufruf von `malloc` zu beschaffen und diesen Speicher dann in einer Schleife aufzuteilen, wie in einem früheren Beispiel bereits gesehen:

```
#include <stdlib.h>

static int anzahl = 0;
```

```

/*= Matrixallokation =====*/
double **matrixalloc(unsigned int n, unsigned int m) {
    double **a;

    a = (double **) malloc(n * sizeof (double *));
    if (a != NULL) {
        a[0] = (double *) malloc(n * m * sizeof (double));
        if (a[0] == NULL) {
            free(a);
            a = NULL;
        }
        else {
            unsigned int i;
            for (i = 1; i < n; i++)
                a[i] = a[i-1] + m;

            anzahl++;
            /* Matrix bereitgestellt */
        }
    }
    return a;
}

/*= Matrixfreigabe =====*/
void matrixfree(double **a) {
    if (a != NULL)
        free(a[0]);

    free(a);
    anzahl--;
    /* Matrix freigegeben */
}

/*= Matrix-Zaehler =====*/
int matrixanzahl(void) {
    return anzahl;
}
/* falls matrixanzahl() != 0, */
/* wurden nicht alle */
/* Matrizen freigegeben */

```

Quelltext 13.2: Matrizen mit beliebigen Dimensionen bereitstellen

Beachte: Zeilenvertauschungen durch Vertauschen von Zeigern sind jetzt ohne weiteres nicht mehr möglich. Zum Freigeben der Matrizen wird vorausgesetzt, dass der Zeiger auf die erste Zeile der Matrix auch auf den Anfang des verwendeten Speicherbereichs zeigt. Sollte dies nach Zeilenvertauschungen nicht mehr der Fall sein, scheitert die Freigabe des Speichers mit `free`. Eine Lösung wäre, in `matrixfree` den kleinsten Zeiger zu suchen und als Argument von `free` zu verwenden.

Auch kann und sollte man in die Funktionen gleich noch eine halbwegs komfortable Behandlung von Fehlern bei der Speicherbereitstellung einbauen, so wie es hier gemacht wurde. Mit der Funktion `matrixanzahl` steht eine einfache Möglichkeit bereit, um zu prüfen, ob bei Programmende alle Matrizen ordnungsgemäß freigegeben wurden.